

Phase 0

Environment & Skeleton

Docker · WSL2 · Qdrant · n8n · FastAPI · Uvicorn · Virtual Environments · Vector-Dimension Discipline

Today's win in one line: You now have a running RAG-ready backend on your laptop — Qdrant as a vector DB, n8n for automation, and a FastAPI app with a health check — plus a Qdrant collection locked at 768 dimensions ready for embeddings. Every choice you made today has a reason behind it, and this guide is where those reasons live.

What you built / verified	How
Docker Desktop on WSL2 backend	Installer + one-click Continue without sign-in; verified with <code>docker run hello-world</code> .
Qdrant vector DB (Docker, port 6333)	<code>docker run -d</code> with named volume; verified via curl + web dashboard.
n8n automation platform (Docker, port 5678)	<code>docker run -d</code> with env vars for TZ + payload limit; owner account created.
Python virtual environment (<code>.venv</code>)	<code>python -m venv</code> , activate script, isolated deps.
FastAPI app + <code>/health</code> route	<code>app/main.py</code> , run with uvicorn, auto-Swagger at <code>/docs</code> .
Qdrant collection <code>content_chunks</code>	<code>PUT /collections/content_chunks</code> with <code>size:768</code> , <code>distance:Cosine</code> .
Git baseline + <code>.gitignore</code> discipline	Project scaffold committed; secrets, venv, vector data, uploads all excluded.

Companion docs: AI Ingestion Engine Build Plan · AI Ingestion Engine Decisions

1 Docker & WSL2 — the foundation of everything today

1.1 What Docker actually is

DEFINITION Docker packages an application together with its OS libraries and dependencies into a portable unit called a **container**. The container runs the same on any machine with Docker, regardless of what's installed on the host.

WHERE IT'S USED Every backend deployed in the last decade: microservices, CI/CD, dev environments, data pipelines. Anywhere "works on my machine" was previously a problem.

EXAMPLE You ran `docker run hello-world`. Docker pulled a tiny image from Docker Hub, started a container from it, ran a program that printed a message, and stopped. Four steps: *CLI* → *daemon* → *image pull* → *container run*.

KEY TERMS **Image** = template (read-only snapshot). **Container** = running instance of an image. **Volume** = a folder mount that survives container restarts. **Port mapping** = expose a container's port to your host machine.

WHY FOR THE INTERNSHIP Your instructor said "n8n through Docker" for a reason: it lets you control payload limits, env vars, credentials, and networking cleanly. You use the same pattern for Qdrant, n8n, and eventually FastAPI.

1.2 Image vs Container — the confusion worth clearing early

Think of an image like a class in code; a container is an instance. The image (`qdrant/qdrant:latest`) is a read-only template you download once. When you `docker run` it, Docker starts a new writable container from that image. You can start ten containers from the same image — they don't share state unless you explicitly connect them with volumes or networks.

The image sits in your local Docker Hub cache after the first pull. That's why the second time you `docker run qdrant/qdrant:latest`, there's no downloading — it's already there.

1.3 WSL2 — why Windows needs it

DEFINITION Windows Subsystem for Linux, version 2. A lightweight real Linux kernel Microsoft ships alongside Windows.

WHY DOCKER USES IT Docker runs Linux containers. Windows can't natively run Linux, so Docker Desktop uses WSL2 as its Linux engine underneath — you never touch it directly, but everything runs on it.

YOU ALREADY HAD IT `wsl --status` showed WSL2 as default with Ubuntu installed. You skipped the whole install-and-reboot dance.

WHY FOR THE INTERNSHIP If asked "how does Docker run on Windows," this is the answer — one word: WSL2. It's why containers feel native despite Windows being a different OS.

1.4 The four flags on every `docker run` today

You ran the same command shape twice (Qdrant, n8n). The same flags will show up in every service you launch:

- `-d` — **detached**. Runs in background, gives you your prompt back.
- `--name X` — **readable handle**. Refer to the container as `X` instead of a random hash.
- `-p host:container` — **port mapping**. Expose the container's port on your machine. Direction matters: host on the left.
- `-v hostpath:containerpath` — **volume mount**. Persist data to your host filesystem. Without this, the container's data vanishes on delete.
- `-e KEY=VALUE` — **environment variable**. Configure the app without editing files inside the container.

WHY VOLUMES ARE NON-OPTIONAL

Your `qdrant_data/` and `n8n_data/` folders are those volumes. If you'd skipped them, every restart of the container would give you a fresh empty database. Your embeddings and workflows would evaporate silently.

2 Qdrant — the vector database

2.1 What a vector database is

DEFINITION A database that stores high-dimensional numeric vectors (embeddings) and lets you find the ones most similar to a given query vector, very fast.

WHERE USED Semantic search, RAG systems, recommendation engines, image similarity, anti-fraud pattern matching — anywhere "find things like this thing" is the operation.

EXAMPLE You embed the sentence "revenue increased in Q2" → 768 numbers. You embed a user's query "how did income change last quarter?" → another 768 numbers. Cosine similarity between them is very high even though they share almost no words. That's semantic search.

KEY TERMS **Collection** = a table analog. **Point** = one stored record (vector + payload). **Payload** = arbitrary JSON metadata attached to a point. **Distance metric** = how similarity is measured (Cosine for text).

WHY FOR THE INTERNSHIP RAG is retrieve-then-generate. Qdrant is the "retrieve" half. Without it, an LLM is just a fancy guesser; with it, answers can cite page numbers and timestamps.

2.2 The vector-dimension footgun (the most important thing today)

When you created the collection, you set `"size": 768`. That number is **permanent** for the collection. Every vector you insert must be exactly 768 floats — no more, no less. Change your embedding model to one that outputs 384-dim vectors and every insert fails.

THE RULE

Model dimension = collection dimension. Pick the model first, then create the collection with that number. Reversing the order or mixing models silently breaks retrieval.

This is why `EMBEDDING_MODEL=nomic-embed-text` and `EMBEDDING_DIM=768` get pinned in `.env`. It's your single source of truth for both sides.

2.3 Cosine similarity — the metric behind semantic search

You picked `"distance": "Cosine"`. Cosine similarity measures the *angle* between two vectors, not their length. Two documents about the same topic point in roughly the same direction in embedding space; the cosine of the angle between them is close to 1. Two unrelated documents point in different directions; cosine is close to 0. Opposite meanings would be closer to -1.

For text embeddings this is the right choice because embedding magnitudes carry mostly noise (document length, batch effects) while direction carries meaning. Dot product and Euclidean distance are alternatives — they matter for other embedding types, rarely text.

3 n8n — the automation layer

3.1 What n8n is and why it's in this project

DEFINITION An open-source workflow automation platform. You build workflows visually (nodes connected on a canvas) that trigger on events and call services.

WHERE USED Automating repetitive backend work — "when a file lands, run this API; if the API returns X, email the team; also update this sheet." No custom glue code needed.

EXAMPLE (YOURS, COMING LATER) Webhook Trigger → HTTP Request to your FastAPI `/ingest/pdf` → Respond to Webhook. That's the whole workflow that will fulfill your instructor's "n8n trigger API endpoint" note.

KEY TERMS **Node** = one step (trigger, action, transform). **Workflow** = a graph of nodes. **Credential** = stored auth (API keys, etc.). **Execution** = one run of a workflow.

WHY FOR THE INTERNSHIP Your instructor explicitly said n8n integration would help you in later tasks. It's the automation layer that *calls* your backend, keeping heavy work in FastAPI and orchestration in n8n.

3.2 The env vars you set today — the "accordingly" part

You passed four env vars when starting n8n. Each has a real reason:

- `GENERIC_TIMEZONE=Asia/Beirut` + `TZ=Asia/Beirut` — schedules and timestamps use your local time. Without this, UTC — subtle bug for any time-of-day trigger.
- `N8N_PAYLOAD_SIZE_MAX=64` — raises the webhook payload cap from the default. **This is the exact "environment variables accordingly" your instructor mentioned** — new n8n versions restrict payload sizes by default; you lift it deliberately.
- `N8N_SECURE_COOKIE=false` — n8n's login cookie is HTTPS-only by default. On plain-HTTP localhost that cookie won't stick and you'd get stuck at the login screen. Localhost dev only; never in production.

WHY THIS PATTERN MATTERS

All three fixes live in Docker `-e` flags, not inside code or config files. That's the point of running services in Docker: environment is configuration, not code changes.

4 FastAPI + Uvicorn + Swagger

4.1 The three-piece pattern (framework + server + docs)

You installed and ran three things that are easy to confuse. They have distinct roles:

- **FastAPI** — the framework. Route decorators, validation, dependency injection, response models. It's just Python; it does nothing on its own until something serves it.
- **Uvicorn** — an ASGI web server. Actually listens on a TCP port, accepts HTTP, and calls your FastAPI code. Without Uvicorn (or another ASGI server) FastAPI is inert.
- **Swagger UI at `/docs`** — auto-generated by FastAPI from your type hints and docstrings. Nothing you wrote by hand. Every route you add gets documented for free.

WHY FOR THE INTERNSHIP Your roadmap says "3 FastAPI + DB projects." This IS that stack. Same pattern will run every future endpoint (ingest, search, ask).

4.2 Reading your `/health` route line by line

```
from fastapi import FastAPI

app = FastAPI(title="AI Ingestion Engine", version="0.1.0")

@app.get("/health")
def health():
    return {"status": "ok"}
```

- `FastAPI(...)` — creates the app instance. The kwargs power Swagger.
- `@app.get("/health")` — decorator that registers the function below as a GET route on `/health`.
- `return {"status": "ok"}` — Python dict, auto-serialized to JSON with content-type `application/json`. FastAPI does that; you don't call `json.dumps`.

4.3 What the uvicorn command actually says

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

- `app.main:app` — module path + object name. "In the file `app/main.py`, use the object `app`."
- `--reload` — watch source files, restart on change. Dev only. Off in production for performance and predictability.
- `--host 0.0.0.0` — bind to all network interfaces (not just `127.0.0.1`). Matters if you want another container or device on your network to reach it.
- `--port 8000` — dev convention. Standard, no special reason.

5 Python discipline — venv, requirements, .gitignore

5.1 Virtual environments

DEFINITION A `.venv` folder inside the project with its own Python interpreter and package set. Isolated from system Python and from other projects.

WHY Two Python projects wanting different FastAPI versions won't fight if each has its own venv. Reproducible environments and clean installs.

THE (. VENV) PROMPT TAG When you activated it, PowerShell added `( .venv )` at the start of your prompt. That's your visual confirmation that pip/python calls are targeting the isolated env, not the system one.

5.2 requirements.txt — reproducibility contract

`pip freeze > requirements.txt` writes every installed package + version into a text file that's committed to git. Anyone (including future you) can recreate the exact environment with `pip install -r requirements.txt`. Version pinning is what makes the phrase "works on my machine" defensible instead of embarrassing.

5.3 What the .gitignore protects you from

Your `.gitignore` excludes: `__pycache__`, `.venv`, `.env`, `storage/uploads/*`, `storage/frames/*`, `qdrant_data/`, `n8n_data/`. Two reasons:

- **Bloat** — `.venv` and Qdrant/n8n data folders are large (50–500 MB). Git repos should hold code, not binaries.
- **Secrets & PII** — `.env` holds API keys / passwords. `n8n_data/` holds credentials. `storage/uploads/` holds user files. None of that should ever hit a public repo.

6 The Docker networking gotcha (say this if asked)

Different addresses depending on who's calling whom

You have three services now: Qdrant (Docker), n8n (Docker), FastAPI (host). Their addresses depend on where the caller runs:

- FastAPI on your host → Qdrant in Docker → `http://localhost:6333`
- FastAPI in a container (later) → Qdrant in the same Docker network → `http://qdrant:6333`
- Any container → Ollama running on your host → `http://host.docker.internal:11434`

WHY THIS IS #1 IN THE "WHY ISN'T IT CONNECTING?" HALL OF FAME

Copy-pasting `qdrant:6333` from a compose example while running FastAPI on your host will give you *connection refused* and no useful error. It's not broken — it's an address that doesn't exist from your host's point of view.

Q&A Likely interview questions on today's work

Answers written in your own simple voice — the way you'd actually say them out loud, not textbook style.

Question	Your answer
What is Docker, in one sentence?	A way to package an app together with its OS dependencies so it runs the same on any machine that has Docker installed.
What's the difference between an image and a container?	An image is a read-only template — like a class. A container is a running instance of that image — like an object. You can start many containers from the same image.
Why is Docker Desktop on Windows using WSL2?	Docker runs Linux containers, and Windows isn't Linux. WSL2 is a real Linux kernel Microsoft ships with Windows, and Docker Desktop uses it as its engine underneath so containers run natively.
What does <code>-p 6333:6333</code> do?	Port mapping. It exposes the container's port 6333 as port 6333 on my host machine, so I can reach Qdrant at <code>localhost:6333</code> . Left side is the host, right side is the container.
Why did you mount a volume for Qdrant?	Because container filesystems are ephemeral — if I delete the container, everything inside vanishes. The volume mounts a folder from my host into the container so the vector data persists across restarts and rebuilds.
Why did you pick Qdrant?	It's an open-source vector database, one-command Docker setup, has a decent web dashboard, and the client is simple. My instructor's notes named it specifically, so I stayed with it.
Why did you pick 768 as the vector size?	Because I locked <code>nomic-embed-text</code> as my embedding model, and that model outputs 768-dimensional vectors. The collection dimension has to match the model exactly — if it doesn't, every insert fails.
What if you needed to change embedding models later?	I'd have to rebuild the collection with the new dimension and re-ingest everything. That's why picking the model up front matters — it's a decision that's expensive to change.
Why Cosine similarity for text?	Because it measures the angle between vectors, not their magnitude, and for text embeddings the direction carries meaning while the magnitude is mostly noise. Two documents about the same topic point the same way in embedding space.
What's the point of running n8n through Docker specifically?	My instructor said newer n8n versions restrict some capabilities by default, so I need env vars to lift payload limits, control cookies, and set the timezone. Docker lets me set all that with <code>-e</code> flags cleanly.
Why FastAPI over Flask or Django?	Async support out of the box, automatic Swagger docs from type hints, and Pydantic validation is built in. For a service that's mostly JSON endpoints, it's the shortest path to a clean API.

What's the difference between FastAPI and Uvicorn?	FastAPI is the framework — it defines my routes and validation. Uvicorn is the actual web server that listens on a port and calls into FastAPI. Without a server like Uvicorn, my FastAPI code doesn't do anything.
Why do you use a virtual environment?	To keep this project's dependencies isolated from my system Python and from other projects. Different projects can want different library versions, and venvs stop them from fighting.
What goes in <code>.gitignore</code> and why?	The venv folder, <code>.env</code> file, uploaded user files, and the Docker volume folders. Two reasons: they're either huge, or they contain secrets. Neither belongs in a git repo.
If FastAPI running on your machine can't reach Qdrant in Docker, what's the first thing you check?	The URL. From my host, Qdrant is at <code>localhost:6333</code> . If I wrote <code>qdrant:6333</code> , that's a container-network name — only reachable from another container, not from my host.

Day 1 · Phase 0 complete. Environment, services, and skeleton in place. Next: Phase 1 — PDF text ingestion end to end. Every choice today (the 768 dim, the Docker networking, the venv discipline) is a foundation the next phases will lean on.